UNIVERSIDAD TECNOLÓGICA DE PEREIRA
Programa Tecnología en Desarrollo de Software
Programación para Móviles

PROYECTO I - CALCULADORA ESTILO iOS (ANDROID NATIVO)

Docente: Ing. Edisson Andrés Montes

1. Objetivo

Desarrollar una aplicación Android nativa con Kotlin y Jetpack Compose que reproduzca una calculadora inspirada en la interfaz de iOS, permita realizar operaciones aritméticas básicas, conserve un historial de operaciones y permita navegar entre ambas pantallas. El proyecto debe evidenciar el uso de MVVM, la separación de responsabilidades y buenas prácticas de programación.

2. Requisitos funcionales

Interfaz de calculadora

- Mostrar botones numéricos del 0 al 9 y las operaciones +, −, ×, ÷, %, y cambio de signo (+/−).
- Incluir las acciones AC (limpiar) y = (calcular).
- Mostrar de forma legible el número ingresado y el resultado o estado actual de la operación.
- Mantener una distribución visual consistente con la referencia de calculadora estilo iOS y adaptar razonablemente los elementos al tamaño de pantalla.

Lógica de cálculo

- Permitir operaciones consecutivas según el comportamiento definido para la calculadora.
- Procesar números decimales correctamente y evitar separadores o conversiones inconsistentes.
- Prevenir errores como división por cero y entradas inválidas; informar el estado de forma comprensible, sin cerrar la aplicación.
- Definir y mantener un comportamiento coherente para AC, cambio de signo, porcentaje y pulsaciones repetidas de =.

Historial

- Mostrar las operaciones realizadas en una lista, incluyendo expresión y resultado (por ejemplo: 5 + 3 = 8).
- Actualizar el historial después de completar una operación válida.
- Conservar el orden de las operaciones y mostrar un estado vacío cuando todavía no existan registros.
- Navegación y estado
- Permitir navegar de la calculadora al historial y regresar sin comportamientos inesperados.

- Mantener el estado de la interfaz sincronizado con el ViewModel y actualizar la pantalla cuando cambie.

3. Requisitos técnicos y buenas prácticas

- Kotlin + Jetpack Compose: Construir la interfaz con Jetpack Compose. No utilizar layouts XML ni librerías externas de UI. Se permiten archivos XML de recursos en res/values cuando se utilicen para cadenas o colores; esta excepción es necesaria para usar stringResource() y colorResource().

- Arquitectura MVVM: Separar la interfaz (Composable), el estado y las acciones de presentación (ViewModel) y la lógica de cálculo (Model). Los Composables no deben ejecutar cálculos de negocio ni contener reglas de operación.

- Estado observable: Exponer desde el ViewModel un estado observable (State) y recolectarlo correctamente desde Compose. La UI debe representar el estado y enviar eventos al ViewModel.

- Recursos de texto: Todo texto fijo visible para el usuario —títulos, botones, mensajes de error, etiquetas y estados vacíos— debe declararse en res/values/strings.xml y consumirse desde Composables mediante stringResource(R.string.nombre_recurso). No repetir textos literales en distintos archivos.

- Recursos de color: Los colores definidos en res/values/colors.xml deben consumirse mediante colorResource(R.color.nombre_recurso) cuando corresponda.

- ViewModel y recursos: No llamar stringResource() ni colorResource() desde el ViewModel: son APIs de Compose y corresponden a la capa de interfaz. El ViewModel debe exponer estados semánticos (por ejemplo, CalculatorError.DivisionByZero) o datos, y la UI debe traducirlos a recursos con stringResource(). Evitar almacenar Activity, View o Context en el ViewModel.

- Código limpio: Usar nombres descriptivos en inglés, mantener una convención consistente, evitar abreviaturas confusas y eliminar código comentado, duplicación y variables globales innecesarias.

- Funciones y componentes: Crear funciones y Composables pequeños con una responsabilidad clara. Extraer componentes reutilizables cuando reduzcan duplicación y mejoren la legibilidad; evitar abstraer sin necesidad.

- Organización: Organizar archivos por responsabilidad y mantener separadas las pantallas, componentes de UI, ViewModel, modelos y lógica de cálculo. La estructura debe ser fácil de comprender y mantener.

- Manejo de errores: Validar los casos límite y representar errores de forma controlada. No ocultar fallos con valores arbitrarios ni permitir que una operación inválida provoque un cierre inesperado.

- Git y entrega: Publicar el proyecto completo en un repositorio Git. Incluir un .gitignore apropiado para Android Studio y no subir archivos generados, credenciales, claves ni contenido local innecesario.

Calificación	
Criterio	**Puntaje Máximo**
Lógica	25
MVVM	25
Código limpio	15
Interfaz	15
Navegación	10
Historial	10
Total	**100 pts**